Facultatea de Electronică,
Telecomunicații și
Tehnologia Informației

DESIGN AND IMPLEMENTATION OF A SERIAL-BASED ULTRASONIC DISTANCE MONITORING SYSTEM

Year 3

Group 2331

Student: Kelemen Dariana

Coord: Prof. dr. ing. Mircea Giurgiu

Contents

1. INTRODUCTION 3
1.1 Theme 3
1.2 Description of Requirements 3
1.3 Solution approach 3
1.4 New Aspects 3
2. THEORETICAL BACKGROUND 4
2.1 Serial Communication (UART) 4
2.2 The Distance Module Logic 4
2.3 Processing the Data 4
3. IMPLEMENTATION 5
3.1 Hardware Implementation 5
3.2 Software implementation 5
4. CONCLUSIONS & BIBLIOGRAPHY 8
4.1 Conclusions 8
4.2 Bibliography 8

1. INTRODUCTION

1.1 Theme: The project involves designing a monitoring system that interfaces a microcontroller (Arduino Nano) with an intelligent ultrasonic module via Serial communication. The system interprets ASCII data streams to visualize distance zones (Safe, Warning, Danger) using a traffic-light LED system.

1.2 Description of Requirements: The requirement was to build a microcontroller system that reads data from a sensor using serial communication. The system needs to process this data (which comes in as text) and use it to control output pins, in this case, three LEDs.

1.3 Solution approach: Usually, ultrasonic sensors like the HC-SR04 use "Trig" and "Echo" pins, which require measuring the time length of a pulse. This approach can be complicated to code and consumes timer resources. To solve this, a special "Distance Display Module" was attached to the sensor. This module calculates the distance automatically and sends the result as a simple text string via a TX pin, making the code simpler and more stable.

1.4 New Aspects: A key part of this project is using "Software Serial." Since the Arduino Nano only has one hardware serial port (which is used by the USB cable to talk to the computer), I had to emulate a second serial port on pins D7 and D8. I also implemented a function to "clean" the received text, removing any letters so that I can use the numbers for the logic.

2. THEORETICAL BACKGROUND

2.1 Serial Communication (UART): UART (Universal Asynchronous Receiver-Transmitter) is a way for devices to talk to each other using two wires: TX (Transmit) and RX (Receive). In my project, the sensor module acts as the transmitter. It sends the distance as a series of characters (ASCII) one by one. The Arduino acts as the receiver.

2.2 The Distance Module Logic: Standard ultrasonic sensors require the microcontroller to send a trigger pulse and measure the echo time. The module I used has its own chip that does this math. It outputs the final distance in centimeters as text (e.g., sending the characters '2', '0', 'c', 'm'). This means the Arduino just has to read the text stream instead of measuring microseconds.

2.3 Processing the Data: Since the sensor sends text (Strings), the Arduino cannot directly compare it to a number like 10 or 20. The software needs to receive the characters, store them in a buffer, and then filter out everything that isn't a digit. Once we have a clean number, we can use simple if/else statements to turn on the correct LED based on the safety thresholds.

3. IMPLEMENTATION

3.1 Hardware Implementation

Components used:

- Arduino Nano V3: The brain of the project.
- HC-SR04 + Distance Display Module: For measuring distance via Serial.
- 3 LEDs: Red, Yellow, and Green to show the zones.
- 3 Resistors (220 Ohm): To protect the LEDs from burning out.
- Breadboard & Wires: To connect everything together.

How it is connected: I placed the Arduino Nano on the breadboard. The sensor module is connected with its TX pin going to the Arduino's D8 pin. I defined D8 as "RX" (Receive) in my code. The LEDs are connected to digital pins D2, D3, and D4. Each LED is connected in series with a 220 Ohm resistor to the Ground (GND) rail to limit the current.

3.2 Software implementation

The code uses the SoftwareSerial library to create a communication port on pins 7 and 8. Below is the C++ code used for the project:

```
#include <SoftwareSerial.h>

// Defining the pins for the LEDs
#define LED_VERDE 2
#define LED_GALBEN 3
#define LED_ROSU 4

// Setting up the software serial port.
// Pin 8 is RX (receives data from sensor), Pin 7 is TX.
SoftwareSerial ultrasonicSerial(8, 7);

String distantaText = "";
```

```
void setup() {
  // Start the serial monitor to see data on the PC
  Serial.begin(9600);

  // Start communication with the sensor module
  ultrasonicSerial.begin(9600);

  // Set LED pins as outputs
  pinMode(LED_VERDE, OUTPUT);
  pinMode(LED_GALBEN, OUTPUT);
  pinMode(LED_ROSU, OUTPUT);
}

void loop() {
  // Check if there is data coming from the sensor
  while (ultrasonicSerial.available()) {
    char c = ultrasonicSerial.read();

    // The sensor sends a 'newline' character at the end of a reading
    if (c == '\n') {

      // Print the raw text to the Serial Monitor for debugging
      Serial.print("Text brut primit: ");
      Serial.println(distantaText);

      // Convert the text to a number (integer)
      int distanta = extrageDistanta(distantaText);
      Serial.print("Distanta extrasa: ");
      Serial.println(distanta);

      // Update the LEDs based on the distance
      actualizeazaLEDuri(distanta);

      // Clear the text variable for the next reading
      distantaText = "";
    }
    else if (c != '\r') {
      // If it's not the end of the line, add the character to our text
string
      distantaText += c;
    }
  }
}

// Function to remove letters and keep only numbers
int extrageDistanta(String text) {
  String numar = "";
```

```
  for (int i = 0; i < text.length(); i++) {
    if (isDigit(text[i])) {
      numar += text[i];
    }
  }
  return numar.toInt();
}

// Function to turn LEDs on/off based on distance
void actualizeazaLEDuri(int distanta) {
  if (distanta > 20) {
    // Far away: Green
    digitalWrite(LED_VERDE, HIGH);
    digitalWrite(LED_GALBEN, LOW);
    digitalWrite(LED_ROSU, LOW);
  } else if (distanta > 10) {
    // Medium distance: Yellow
    digitalWrite(LED_VERDE, LOW);
    digitalWrite(LED_GALBEN, HIGH);
    digitalWrite(LED_ROSU, LOW);
  } else {
    // Too close: Red
    digitalWrite(LED_VERDE, LOW);
    digitalWrite(LED_GALBEN, LOW);
    digitalWrite(LED_ROSU, HIGH);
  }
}
```

4. CONCLUSIONS & BIBLIOGRAPHY

4.1 Conclusions:

In this project, I successfully connected an Arduino Nano to a serial ultrasonic sensor. Using the external display module made the wiring and coding much easier compared to the standard Trig/Echo method. The string processing function works well to clean up the data, and the LEDs respond immediately to changes in distance. The thresholds of 10cm and 20cm provide a clear visual warning system.

4.2 Bibliography

- HC-SR04 Ultrasonic Sensor Datasheet SparkFun Electronics https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf
- UART Serial Communication Standard Arduino Official Documentation https://docs.arduino.cc/learn/communication/uart/
- Arduino SoftwareSerial Library Limitations & Usage https://docs.arduino.cc/learn/built-in-libraries/software-serial
- Parsing ASCII Data Streams in C++ Arduino Documentation https://www.arduino.cc/en/Tutorial/ReadASCIIString
- Arduino Nano Technical Reference Arduino Official Documentation. https://docs.arduino.cc/hardware/nano